

Notebook Run Order And Config Guide

Google Colab workflow, parallel run rules, and config search spaces

This guide explains which Google Colab notebooks to run first, which ones can run at the same time, and which config values to try when searching for the best AI Chessathon bot.

It is written for a teammate who does not need a computer science background.

Project Location

In Google Colab, use this project root:

```
from pathlib import Path
PROJECT_ROOT = Path("/content/drive/MyDrive/Colab Notebooks/Education/Deep
Reinforcement Learning/Chess")
```

Everything should be saved under PROJECT_ROOT.

Expected folder layout:

The Short Version

Run this first:

Then these can run after Notebook 1:

Then run:

Run this last only after training and evaluation:

Notebook Order

1. Environment And Encoding

Filename:

Run first.

What it does:

- Sets up the exact chess environment.
- Uses FEN positions.
- Uses python-chess for legal moves and rules.
- Defines board encoding.
- Defines move/action encoding.
- Creates the shared chess_rl/ helper code.

What it produces:

Do not start training until this exists.

2. Supervised Policy/Value Training

Filename:

Run after Notebook 1.

What it does:

- Trains the policy network from labelled positions.
- Trains the value network from game results or evaluation labels.
- Saves model checkpoints.

What it produces:

This is usually the first serious training notebook.

3. Self-Play / League Training

Filename:

Run after Notebook 1.

What it does:

- Makes bots play games against themselves.
- Plays against older checkpoints or baselines.
- Produces extra training data and stronger candidate models.

What it produces:

This works better after Notebook 2 has produced a first model, but it can also start from the classical tree-search agent.

4. Evaluation And Comparison

Filename:

Run after at least one candidate model or agent exists.

What it does:

- Compares candidates.
- Runs games against baselines and previous versions.
- Reports wins, draws, losses, crashes, timeouts, average move time, and model inference time.

What it produces:

Use this notebook to decide which model is currently best.

5. Final Export And Compliance

Filename:

Run last.

What it does:

- Selects the final candidate.
- Exports CPU-safe model files.
- Checks submission size.
- Checks final runtime constraints.
- Prepares submission_candidate/.

What it produces:

Important: do not run final compliance checks repeatedly during Notebooks 1-4. Compliance checks happen here after training and evaluation are complete.

What Can Run In Parallel

Parallel means two Colab sessions, two teammates, or two GPUs can run jobs at the same time.

Safe parallel options after Notebook 1:

Safe parallel options after models exist:

Examples:

Each parallel run must write to its own folder.

Good:

Bad:

What Must Run Sequentially

These must happen in order:

Dependency map:

Notebook 5 should not run before training and evaluation have produced a candidate worth exporting.

Recommended Team Split

If several people are helping:

If one person is working:

CUDA / GPU Rules

Use CUDA/GPU for training:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

Good places to use GPU:

Final competition runtime must be CPU:

```
device = "cpu"  
torch.set_num_threads(1)
```

Notebook 5 must verify that final exported artifacts can load and run on CPU.

Config Files To Create

Put config files here:

Recommended config files:

Use one config per experiment run. Do not edit one config while a notebook using it is still running.

Suggested Starter Configs

supervised_small.yaml

Use this first to confirm training works.

```
model:
  residual_blocks: 4
  channels: 64
  value_head_hidden: 128
  activation: relu
  use_batch_norm: true
  dropout: 0.0

training:
  epochs: 10
  batch_size_cuda: 512
  batch_size_cpu: 64
  learning_rate: 0.0003
  weight_decay: 0.0001
  value_weight: 1.0
  scheduler: cosine
  early_stopping_patience: 4
```

supervised_medium.yaml

Use this as the main baseline training run.

```
model:
  residual_blocks: 6
  channels: 128
  value_head_hidden: 256
  activation: relu
  use_batch_norm: true
  dropout: 0.0

training:
  epochs: 20
  batch_size_cuda: 1024
  batch_size_cpu: 128
  learning_rate: 0.0003
  weight_decay: 0.0001
  value_weight: 1.0
  scheduler: cosine
  early_stopping_patience: 4
```

supervised_wide.yaml

Use this if GPU memory is available and the medium model is not strong enough.

```
model:
  residual_blocks: 8
  channels: 192
  value_head_hidden: 512
  activation: gelu
  use_batch_norm: true
  dropout: 0.05

training:
  epochs: 30
  batch_size_cuda: 1024
  batch_size_cpu: 128
  learning_rate: 0.0001
  weight_decay: 0.0001
  value_weight: 1.0
  scheduler: one_cycle
  early_stopping_patience: 5
```

self_play_fast.yaml

Use this for quick self-play experiments.

```
self_play:
  games_per_iteration: 128
  max_iterations: 5
  temperature_early: 1.0
  temperature_late: 0.1
  temperature_cutoff_ply: 12
  opponent_pool: latest_and_previous

search:
  alpha_beta_max_depth: 3
  quiescence_depth: 2
  policy_top_k_root: 16
  policy_top_k_internal: 8
  value_eval_mix: 0.5
```

self_play_deeper_search.yaml

Use this for slower but stronger self-play.

```
self_play:
  games_per_iteration: 512
  max_iterations: 10
  temperature_early: 1.0
  temperature_late: 0.05
  temperature_cutoff_ply: 16
  opponent_pool: league

search:
  alpha_beta_max_depth: 4
  quiescence_depth: 4
  policy_top_k_root: 24
  policy_top_k_internal: 16
  value_eval_mix: 0.5
```

evaluation_quick.yaml

Use this to remove weak candidates quickly.

```
evaluation:
  opponents:
    - baselines/greedy
    - current_classical_agent
  games: 16
  base_ms: 10000
  increment_ms: 100
```

evaluation_serious.yaml

Use this before choosing a final candidate.

```
evaluation:
  opponents:
    - baselines/greedy
    - baselines/minimax
    - current_classical_agent
    - previous_best_model
  games: 64
  base_ms: 10000
  increment_ms: 100
```

final_export.yaml

Use only in Notebook 5.

```
export:
  selected_checkpoint: exports/best_model_cpu.pt
  export_format: pytorch_state_dict
  optional_onnx: true
  quantization: none
  force_cpu: true
  cpu_threads: 1
  max_unzipped_submission_mb: 50
```

Hyperparameter Search Space

Put the full search grid in:

Suggested values:

```
model:
  residual_blocks: [4, 6, 8, 10]
  channels: [64, 96, 128, 192]
  value_head_hidden: [128, 256, 512]
  activation: [relu, gelu, silu]
  use_batch_norm: [true, false]
  dropout: [0.0, 0.05, 0.1]

training:
  learning_rate: [0.0001, 0.0003, 0.001]
  weight_decay: [0.00001, 0.0001, 0.0003]
  batch_size_cuda: [512, 1024, 2048, 4096]
  epochs: [10, 20, 30, 50]
  value_weight: [0.25, 0.5, 1.0, 2.0]
  policy_label_smoothing: [0.0, 0.02, 0.05, 0.1]
  scheduler: [cosine, onecycle, step, none]

self_play:
  games_per_iteration: [128, 256, 512, 1024]
  temperature_early: [0.75, 1.0, 1.25]
  temperature_late: [0.0, 0.05, 0.1, 0.25]
  temperature_cutoff_ply: [8, 12, 16, 20]

runtime_search:
  policy_top_k_root: [8, 16, 24, 32, all]
  policy_top_k_internal: [8, 16, 24, all]
  value_eval_mix: [0.0, 0.25, 0.5, 0.75, 1.0]
  alpha_beta_max_depth: [3, 4, 5, 6]
  quiescence_depth: [2, 4, 6]
  time_budget_fraction: [0.02, 0.04, 0.06, 0.08]
  transposition_table_size: [50000, 100000, 200000]

export:
  export_format: [pytorch_state_dict, torchscript, onnx]
  quantization: [none, dynamic_int8]
  cpu_threads: [1]
```

Do not try the full grid at once. It is too large. Pick a few sensible combinations.

Practical Search Strategy

Recommended order:

Most important settings to try first:

Least important at the beginning:

Final-Only Compliance Rule

Compliance checks belong only in:

Do not run full compliance after every training run.

Notebook 5 should check:

Plain-English Summary

Think of the notebooks like this:

Run 01 first. Run 05 last. The middle notebooks can run in parallel only if they save to different folders.

Generated from docs/NOTEBOOK_RUN_ORDER_README.md.